String Matching

UMAMA RAHMAN

String matching

1. **Text Search:**
   - Searching for keywords or phrases in documents or web pages.
2. **DNA Sequence Analysis:**
   - Finding specific gene sequences within a larger DNA strand
3. **Plagiarism Detection:**
   - Comparing student submissions or published papers to detect copied content.
4. **Search Engines:**
   - Matching user queries to relevant web pages.
5. **Network Security:**
   - Detecting malicious patterns in network traffic or log files.
6. **Auto-complete Features:**
   - Providing suggestions based on partial user input in search bars or forms.

Naïve Algorithm for Pattern Matching

- Given **text** string with length **n** and a **pattern** with length **m,** the task is to prints all occurrences of **pattern** in **text.**

Note: You may assume that n > m.

Naive String Matching Algorithm

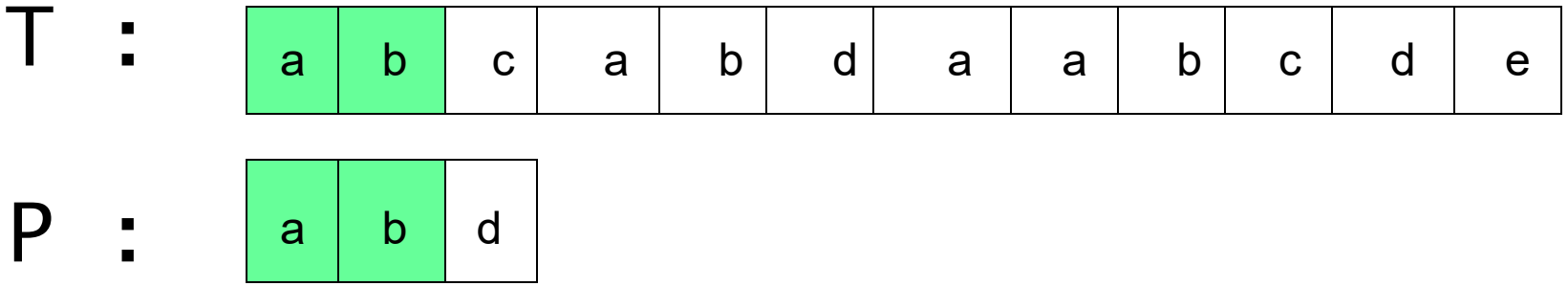
T :

a	b	c	a	b	d	a	a	b	c	d	e

P :

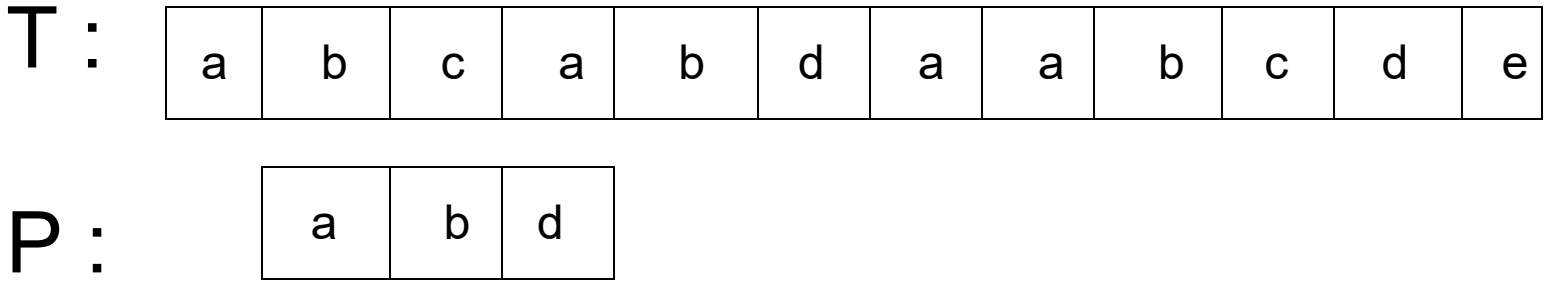
a	b	d

Naive String Matching Algorithm



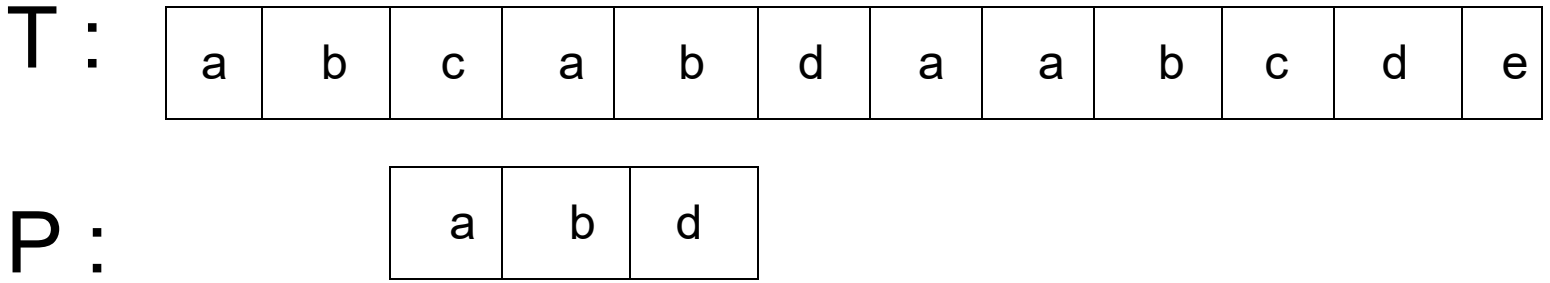
Mismatch after 3 Comparisons

Naive String Matching Algorithm



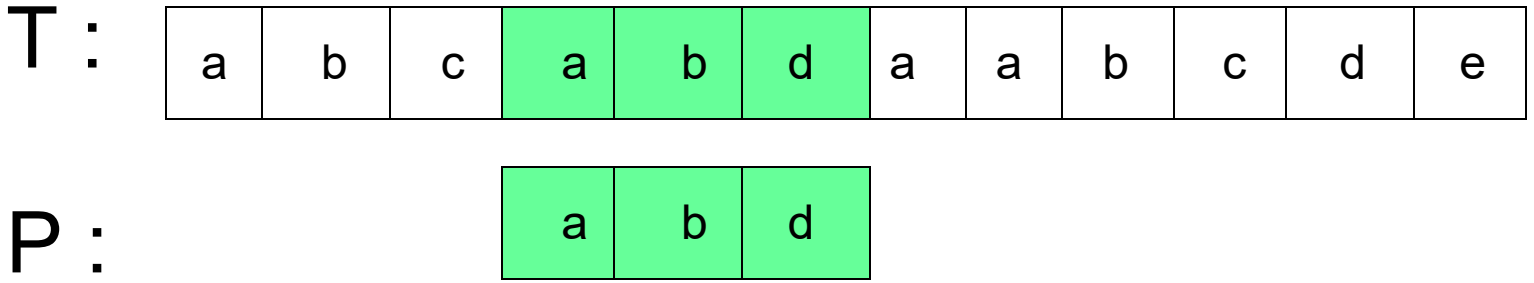
Mismatch after 1 Comparison

Naive String Matching Algorithm



Mismatch after 1 Comparison

Naive String Matching Algorithm



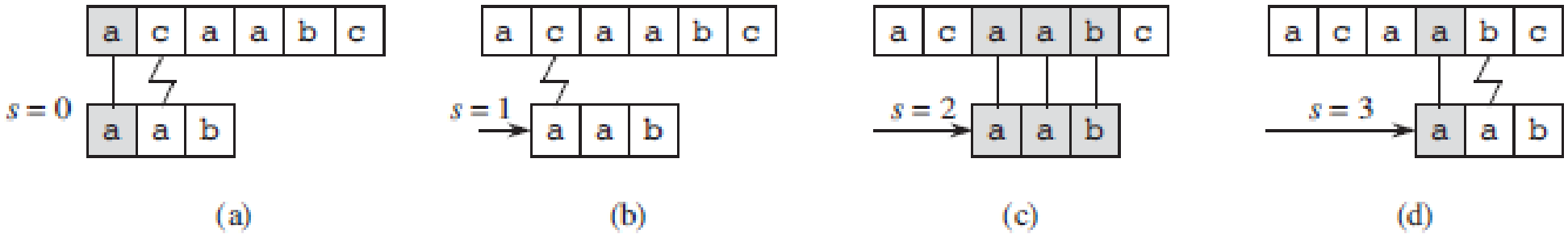
Match found after 3 Comparisons

Thus, after 8 comparisons the substring P is found in T.

Naïve String matching Algorithm

- The naive algorithm finds all valid shifts using a loop that checks the condition P[1 … m] = T[s+1 … s+m] for each of the n-m+1 possible values of s.

```
NAIVE-STRING-MATCHER(T, P)
1  n = T.length
2  m = P.length
3  for s = 0 to n − m
4       if P[1..m] == T[s + 1..s + m]
5            print "Pattern occurs with shift" s
```



Time complexity of Naive String Matching Algorithm

Best Case: O(m)

- When the pattern is found at the very beginning of the text (or very early on).
- The algorithm will perform a constant number of comparisons, typically on the order of O(m) comparisons, where m is the length of the pattern.

Worst Case: $O(n^2)$

- When the pattern doesn't appear in the text at all or appears only at the very end.
- The algorithm will perform O((n-m+1)*m) comparisons, where n is the length of the text and m is the length of the pattern.
- In the worst case, for each position in the text, the algorithm may need to compare the entire pattern against the text.

Rabin-Karp Matching Algorithm

Rabin-Karp Algorithm

Rabin and Karp proposed a string-matching algorithm that performs well in practice. The Rabin-Karp algorithm uses **Θ(m) preprocessing time**, and its worst-case running time is ((n-m+1)*m). H**owever, its average-case running time is better.**

How it works

- Consider a hashing scheme
  - Each symbol in alphabet Σ can be represented by an ordinal value { 0, 1, 2, ..., d }
    - |Σ| = d
    - “Radix-d digits”

How it works

- Hash pattern P into a numeric value
  - Let a string be represented by the sum of these digits
  - Example

    { A, B, C, ..., Z } → { 1, 2, ..., 26 }

    - BAN → 2 + 1 + 14 = 17
    - CARD→ 3 + 1 + 18 + 4 = 26

How it works

a->1
b->2
c->3
d->4
e->5
f->6

Consider pattern, P : adbd

Text, T : adadbdb

length of pattern, m = 4

length of text, n = 7

Total substring = n – m + 1 = 7 - 4 + 1 = 4

Preprocessing

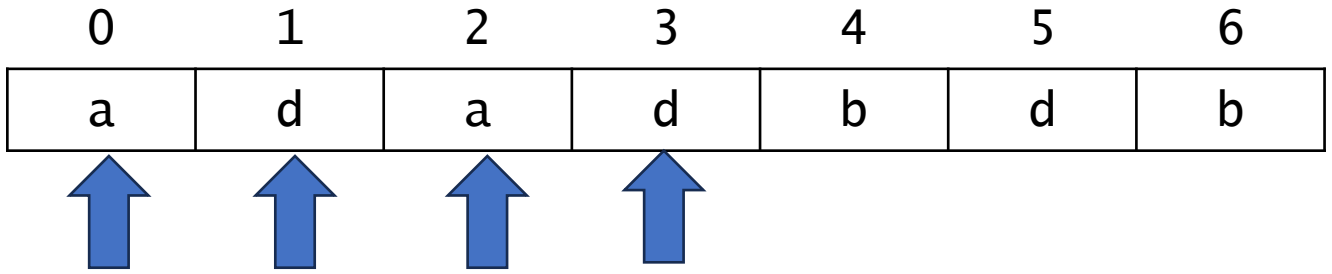
Hash value of pattern (“adbd”) = 1 + 4 + 2 + 4 = 11

Substring of length m starting from s index = T[s…s+m-1]

How it works

Hash value of pattern (adbd) = 11

a->1
b->2
c->3
d->4
e->5
f->6



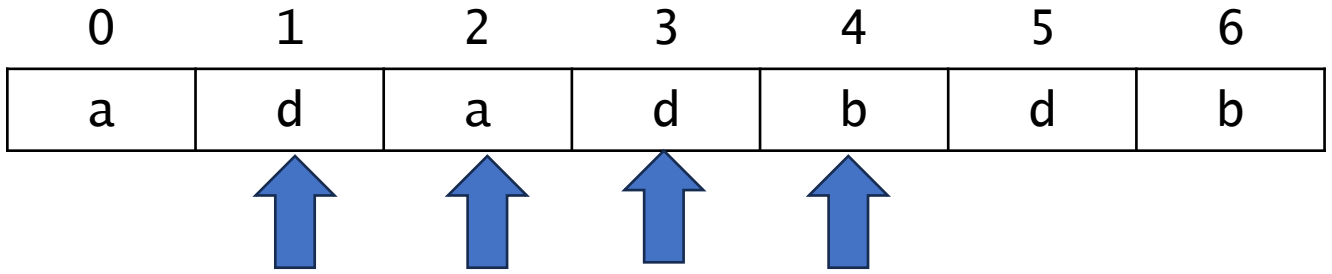
Hash value for this substring = 1+4+1+4=10 -> not matched

i	substring	Hash value	Remark
0	adad	T[0]+T[1]+T[2]+T[3] = 10	Not matched

How it works

Hash value of pattern (adbd) = 11

a->1
b->2
c->3
d->4
e->5
f->6



Hash value for this substring = 4+1+4+2=11 -> hash value matched. Now check the substring with the pattern , if this is actually a match

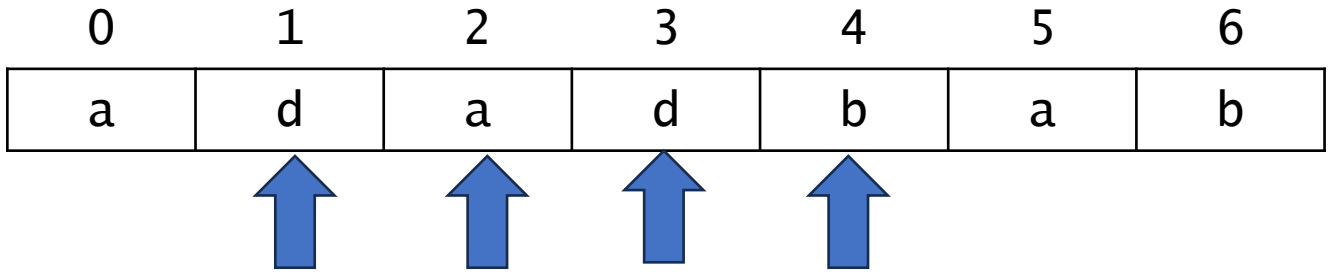
i	substring	Hash value	Remark
0	adad	h = T[0]+T[1]+T[2]+T[3] = 10	Not matched
1	dadb	h = T[1]+T[2]+T[3]+T[4] = 11 or h = 10 – 1 + 2 = 11	

This is called hash value computation using **rolling hash function**

How it works

Hash value of pattern (adbd) = 11

a->1
b->2
c->3
d->4
e->5
f->6



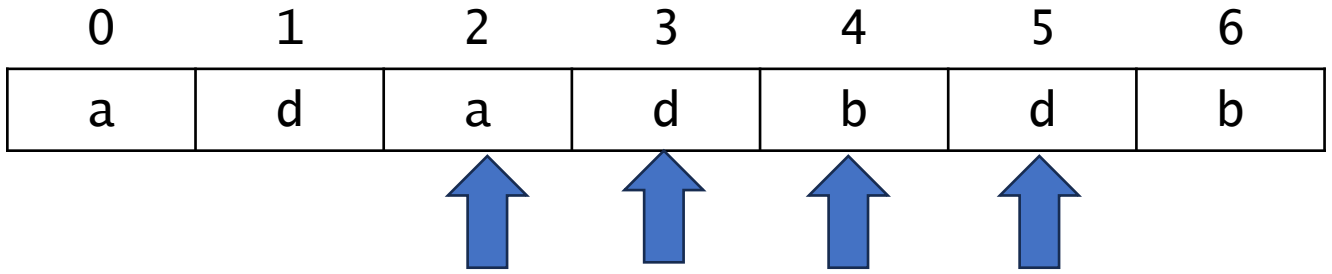
Hash value for this substring = 4+1+4+2=11 -> hash value matched. This is not exact string match. This is called spurious hit.

i	substring	Hash value	Remark
0	adad	h = T[0]+T[1]+T[2]+T[3] = 10	Not matched
1	dadb	h = 10 – 1 + 2 = 11	Spurious hit

How it works

Hash value of pattern (adbd) = 11

a->1
b->2
c->3
d->4
e->5
f->6



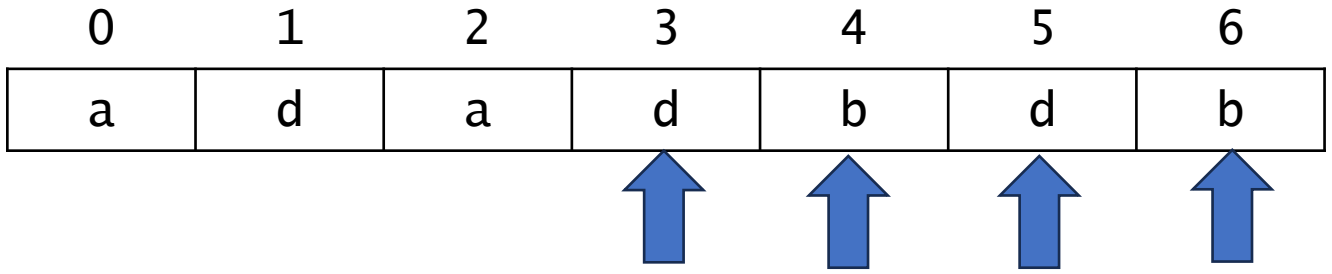
Hash value for this substring = 11 – 4 + 4 = 11 → hash value matched.
Now check the substring with the pattern , if this is actually a match

i	substring	Hash value	Remark
0	adad	h = T[0]+T[1]+T[2]+T[3] = 10	Not matched
1	dadb	h = 10 – 1 + 2 = 11	Spurious hit
2	adba	h = 11 – 4 + 4 = 11	hit/match

How it works

Hash value of pattern (adbd) = 11

a->1
b->2
c->3
d->4
e->5
f->6



Hash value for this substring = 11 – 4 + 2 = 9 → not matched

i	substring	Hash value	Remark
0	adad	h = T[0]+T[1]+T[2]+T[3] = 10	Not matched
1	dadb	h = 10 – 1 + 2 = 11	Spurious hit
2	adba	h = 11 – 4 + 4 = 11	hit/match
3	dbdb	h = 11 – 4 + 2 = 9	Not matched

Rolling hash function

- Rolling hash function is used to compute hash values in Robin Karp algorithm
- Advantage of Rolling hash function
  - It needs less computation
  - Useful for large pattern

Upper limits

- **Problem**
  - For long patterns, or for large alphabets, the number representing a given string may be too large to be practical
- **Solution**
  - Use MOD operation
  - When MOD q, values will be < q
- Example
  - BAN = 2 + 1 + 14 = 17
    - 17 mod 13 = 4
    - BAN → 4
  - CARD = 3 + 1 + 18 + 4 = 26
    - 26 mod 13 = 0
    - CARD → 0

Spurious Hits

- **Question**
  - Does a hash value match mean that the patterns match?
- **Answer**
  - No – these are called “spurious hits”
- Possible cases
  - MOD operation interfered with uniqueness of hash values
    - 14 mod 13 = 1
    - 27 mod 13 = 1
    - MOD value q is usually chosen as a prime
  - Information is lost in generalization (addition)
    - BAN → 2 + 1 + 14 = 17
    - CAM → 3 + 1 + 13 = 17

Rabin-Karp Algorithm

```
RABIN-KARP-MATCHER(T, P, d, q)
 1  n = T.length
 2  m = P.length
 3  h = d^(m-1) mod q
 4  p = 0
 5  t_0 = 0
 6  for i = 1 to m                          // preprocessing
 7      p = (dp + P[i]) mod q
 8      t_0 = (d t_0 + T[i]) mod q
 9  for s = 0 to n - m                      // matching
10      if p == t_s
11          if P[1..m] == T[s + 1..s + m]
12              print "Pattern occurs with shift" s
13      if s < n - m
14          t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) mod q
```

Rabin Karp Algorithm

- T = “8314152”, P = “31415”, $\sum = \{0,1,\ldots,9\}$
  - n = 7, m = 5, d = 10
  - p = 31415
  - $t_0 = 83141$
  - t_{0+1} = 10.(83141-10^{5-1}.T[0+1])+T[0+5+1] = 10.(83141-10^4.8) + 5
    = 31415

Hash function for Rabin-Karp:

$$\text{hash(s)} = \text{s[0]} * \text{d}^{\text{n-1}} + \text{s[1]} * \text{d}^{\text{n-2}} + \ldots\ldots\ldots\ldots\ldots + \text{s[n-1]}*\text{d}^{0}$$

Rolling hash function for computing the hash value of the substrings:

$$t_{s+1} = d(t_s - d^{m-1}T[s+1]) + T[s+m+1]$$

Assuming indexing of the string is started from 1

Rabin-Karp Algorithm

p and t_s may be too large to work with conveniently. If P contains m characters, then we cannot reasonably assume that each arithmetic operation on p (which is m digits long) takes "constant time."



p, t_s may be large, so use mod

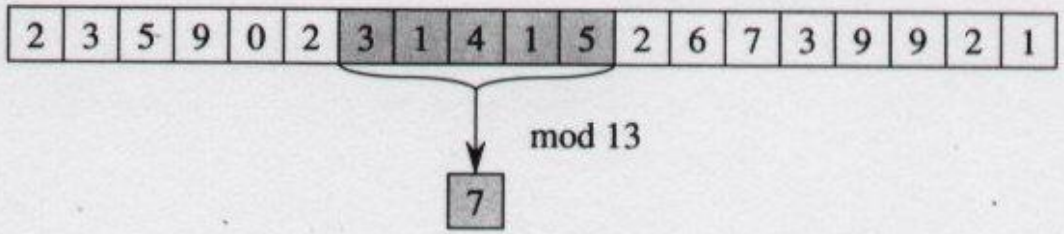


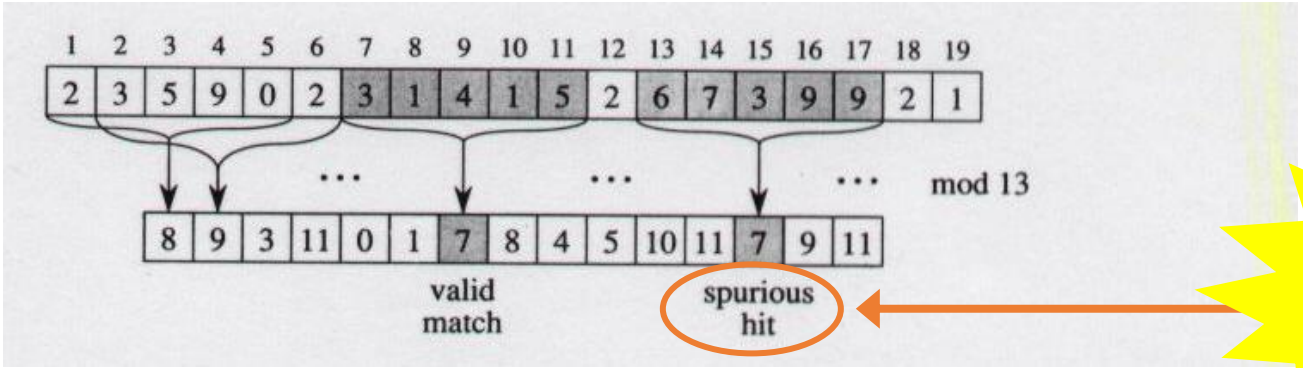
Figure 34.4 The Rabin-Karp algorithm. Each character is a decimal digit, and we compute values modulo 13. **(a)** A text string. A window of length 5 is shown shaded. The numerical value of the shaded number is computed modulo 13, yielding the value 7.

Rabin-Karp Algorithm

h = $d^{m-1} \, (mod \, q)$

t_{s+1} = (d (t_s – h T[s+1]) + T[s+m+1]) mod q

$$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q \,, \qquad (34.2)$$

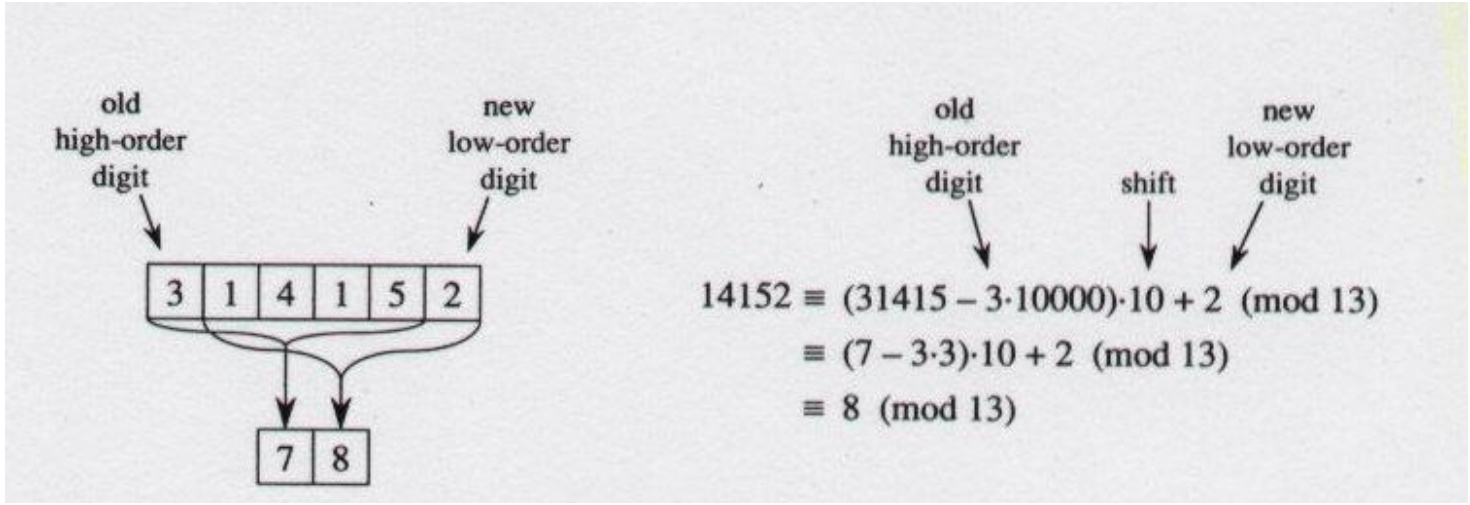


p = 31415

spurious hit

(b) The same text string with values computed modulo 13 for each possible position of a length-5 window. Assuming the pattern $P = 31415$, we look for windows whose value modulo 13 is 7, since $31415 \equiv 7 \pmod{13}$. Two such windows are found, shown shaded in the figure. The first, beginning at text position 7, is indeed an occurrence of the pattern, while the second, beginning at text position 13, is a spurious hit.

Rabin-Karp Algorithm



(c) Computing the value for a window in constant time, given the value for the previous window. The first window has value 31415. Dropping the high-order digit 3, shifting left (multiplying by 10), and then adding in the low-order digit 2 gives us the new value 14152. All computations are performed modulo 13, however, so the value for the first window is 7, and the value computed for the new window is 8.

Rabin Karp Algorithm using modulus

Hash function for Rabin-Karp:

$$\text{hash}(s) = (\, s[0] * d^{m-1} + s[1] * d^{m-2} + \ldots\ldots\ldots\ldots\ldots + s[m-1]*d^{0}\,) \bmod q$$

where, m = length of s

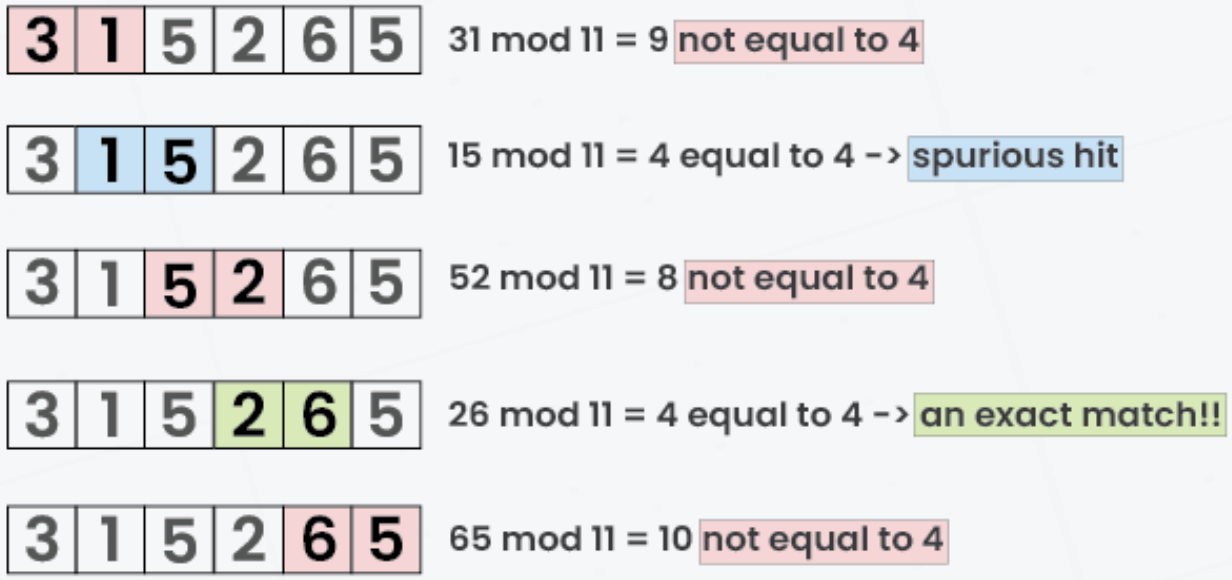
Rolling hash function for computing the hash value of the substrings:

$$t_{s+1} = (d(t_s - T[s] * h) + T[s + m + 1]) mod\ q$$

Where, $h = d^{m-1} \bmod q$

Example

- Given Text = 315265 and Pattern = 26
- We choose b = 11
- P mod b = 26 mod 11 = 4



As we can see, when a match is found, further testing is done to ensure that a match has indeed been found.

Rabin Karp Algorithm

A Rabin-Karp example

- Given T = 31415926535 and P = 26
- We choose q = 11
- P mod q = 26 mod 11 = 4

Here, we are using a window of size 2. It corresponds to the length of the pattern.
For the first window, we see that the value is 31.
If we wanted to find out the value of the next window,
we would merely subtract the high-order digit 3 and add the low-order digit 4,
so that we are left with 14, the value of the second window.



31 mod 11 = 9 not equal to 4



14 mod 11 = 3 not equal to 4



41 mod 11 = 8 not equal to 4

Rabin-Karp example continued



15 mod 11 = 4 equal to 4 -> spurious hit



59 mod 11 = 4 equal to 4 -> spurious hit



92 mod 11 = 4 equal to 4 -> spurious hit



26 mod 11 = 4 equal to 4 -> an exact match!!



65 mod 11 = 10 not equal to 4

Rabin-Karp example continued



53 mod 11 = 9 not equal to 4



35 mod 11 = 2 not equal to 4

***As we can see,
when a match is found,
hash value of substring = hash value of pattern
further testing is done to ensure that a match has indeed been found.

Exercise - 1

- Working with modulo q = 13 and d = 10, find all valid matches and spurious hits (incorrect matches) in the text T = “41591851" for pattern P = “159", using the Rabin-Karp algorithm.

T = 41591851, P = 159, q = 13 , d = 10

n = 8
m = 3
q = 13
d = 10
$d^{m-1} = 10^2 = 100$

Hash value of pattern, p = 159 % 13 = 3

s	substring	Hash value , t	t % q	remark
i=0	**415**91851	t = $4*10^2 + 1*10 + 5 = 415$	415 % 13 = 12	Not matched
i=1	4**159**1851	t = 10*(415 – 4*100) + 9 = 159	159 % 13 = 3	match
i=2	41**591**851	t = 10*(159 – 1*100) + 1 = 591	591 % 13 = 6	Not matched
i=3	415**918**51	t = 10*(591 – 5*100) + 8 = 918	918 % 13 = 8	Not matched
i=4	4159**185**1	t = 10*(918 – 9*100) + 5 = 185	185 % 13 = 3	Spurious hit
i=5	41591**851**	t = 10*(185 – 1*100) + 1 = 851	851 % 13 = 6	Not matched

Exercise - 2

The numeric values of the letters of the product code:

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10

The hash function is defined as follows:

hash(s) = (s[0] * 10^(n-1) + s[1] * 10^(n-2) + … + s[n-1]*10^(n-n)) mod q

Where,

hash(s) is the hash value of string

n is the length of the string

10 is the base of the hash function. q = 131.

You are given a dataset of product codes, for example, "GJABCDFGH," and you need to efficiently identify occurrences of a specific product code pattern within it. The pattern you are looking for is "CDFG". Clearly show the step-by-step calculations for the hash values, modulus operations and the final results.

T = "GJABCDFGH", P = "CDFG" d = 10

hash(s) = (s[0] * 10^(n-1) + s[1] * 10^(n-2) + … + s[n-1]*10^(n-n)) mod q

Length of pattern , n= 4, q = 131

Hash value of pattern, p = hash("CDFG") = $(3*10^3+4*10^2+6*10+7)$ % 131

= 3467 % 131 = 61

s	substring	Hash value , t	t % q	remark
i=0	**GJAB**CDFGH	t = $(7*10^3 + 10*10^2 + 1*10 + 2)$ = 8012	8012 % 131 = 21	Not matched
i=1	G**JABC**DFGH	t = 10 * (8012 – $7*10^3$) + 3 = 10123	10123 % 131 = 36	Not matched
i=2	GJ**ABCD**FGH	t = 10 * (10123 – $10*10^3$) + 4 = 1234	1234 % 131 = 55	Not matched
i=3	GJA**BCDF**GH	t = 10 * (1234 – $1*10^3$) + 6 = 2346	2346 % 131 = 119	Not matched
i=4	GJAB**CDFG**H	t = 10 * (2346 – $2*10^3$) + 7 = 3467	3467 % 131 = 61	match
i=5	GJABC**DFGH**	t = 10 * (3467 – $3*10^3$) + 8 = 4678	4678 % 131 = 93	Not matched

Exercise - 3

Consider the mentioned notations:
Text, t = "NAMANCK"
Pattern, p = "MAN"
Modulo, q = 11
Hash of a string XYZ, $h(XYZ) = (X + Y + Z) \bmod q$;
where X, Y, and Z are the alphabetical sequence of keys [*i.e.: A =1, B=2, C=3, … Y=25, Z=26*]
Show the indices of *valid hit* and *spurious hit* (if any) using the aforementioned hash function **using rolling hash technique.**

T = "NAMANCK", P = "MAN" q = 11

$h(XYZ) = (X + Y + Z) \ mod \ q$

Hash value of pattern, p = hash("MAN") = (13 + 1 + 14) % 11

= 28 % 11 = 6

s	substring	Hash value , t	t % q	remark
i=0	**NAM**ANCK	t = (14 + 1 +13) = 28	28 % 11 = 6	Spurious hit
i=1	N**AMA**NCK	t = (28 – 14 + 1) = 15	15 % 11 = 4	Not matched
i=2	NA**MANC**K	t = (15 – 1 + 14) = 28	28 % 11 =6	match
i=3	NAM**ANC**K	t = (28 – 13 + 3) = 18	18 % 11 =7	Not matched
i=4	NAMA**NCK**	t = (18 – 1 + 11) = 6	28 % 11 = 6	Spurious hit

Indices of valid hit : 2
Indices of spurious hit : 0, 4

Exercise - 4

The numeric values of the letters of the product code:

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10

The hash function is defined as follows:

hash(s) = (s[0] * d^0 + s[1] * d^1+ …… + s[n-1]*d^{n-1}) % q

Where,

hash(s) is the hash value of string

n is the length of the string

d = 7, q = 159.

You are given a dataset of product codes, for example, "GJABCDFGH," and you need to efficiently identify occurrences of a specific product code pattern within it. The pattern you are looking for is "CDFG". Clearly show the step-by-step calculations for the hash values, modulus operations and the final results.

T = "GJABCDFGH", P = "CDFG" d = 7

hash(s) = (s[0] * d^0 + s[1] * d^1+ …… + s[n-1]*d^{n-1}) % q

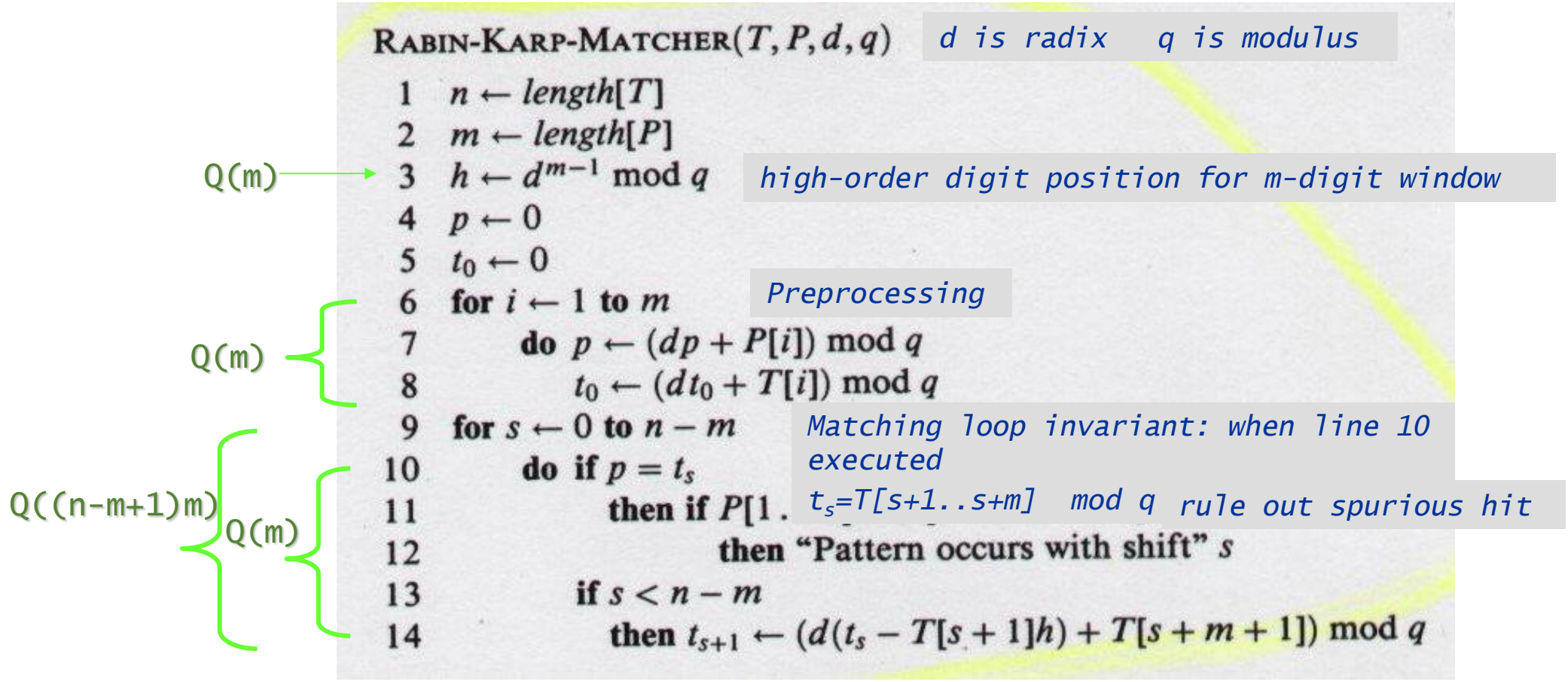
Length of pattern , n= 4, q = 159

Hash value of pattern, p = hash("CDFG") = (3*7^0+4*7^1+6*7^2+7*7^3) % 159

= 2726 % 159 = 23

s	substring	Hash value , t	t % q	remark
i=0	**GJAB**CDFGH	t = (7*7^0 + 10*7^1 + 1*7^2 + 2*7^3) = 812	812 % 159 = 17	Not matched
i=1	G**JABC**DFGH	t = (812 - 7) / 7 + 3 * 7^3 = 1144	1144 % 159 = 36	Not matched
i=2	GJ**ABCD**FGH	t = (1144 - 10) / 7 + 4 * 7^3 = 1534	1534 % 159 = 103	Not matched
i=3	GJA**BCDF**GH	t = (1534 - 1) / 7 + 6 * 7^3 = 2277	2277 % 159 = 51	Not matched
i=4	GJAB**CDFG**H	t = (2277 - 2) / 7 + 7 * 7^3 = 2726	2726 % 159 = 23	match
i=5	GJABC**DFGH**	t = (2726 - 3) / 7 + 8 * 7^3 = 3133	3133 % 159 = 112	Not matched

Rabin-Karp Algorithm



worst-case running time is in O((n-m+1)m)

Time complexity of Rabin Karp

- **Best case:**

→No spurious hit

→Only one substring matches

→Char by char matching is done when hash value matches

Time complexity = Preprocessing (computing hash value of pattern)
+ Time for computing hash value of substrings
+ time for char-by-char comparison

= O(m) + O(m + (n-m)*1) + O(m) = O(n+m) = O(n) ; n>>m

Time complexity of Rabin Karp

- **Worst case:**

→Every substring generates spurious hit

→Char by char matching is done for every substring

Time complexity = Preprocessing (computing hash value of pattern)
+ Time for computing hash value of substrings
+ time for char-by-char comparison
= O(m + (n-m)*1) + O((n-m+1) * m)
= O((n-m+1)*m); same as naïve string matching algorithm